2017 International Symposium ELMAR • IEEE Xplore • Zadar, Croatia DOI: 10.23919/ELMAR.2017.8124460 • pp. 167–171

Improving Optical Character Recognition Performance for Low Quality Images

Matteo Brisinello[1], Ratko Grbić[1], Matija Pul[2], Tihomir Anđelić[2]

[1]Faculty of Electrical Engineering, Computer Science and Information Technology Osijek, J. J. Strossmayer University of Osijek, Croatia

[2]RT-RK Institute for Computer Based Systems, Osijek, Croatia

Email: matteo.brisinello@ferit.hr, ratko.grbic@ferit.hr

Abstract—Optical Character Recognition (OCR) is a widely used technology for converting images containing printed or handwritten text into machine-encoded text. While modern OCR engines achieve high recognition rates on high-resolution scanned documents with homogeneous backgrounds, their accuracy drops significantly when applied to low-quality images characterized by low spatial resolution, complex colored backgrounds, compression artifacts, and uneven illumination. In this paper, four classical image preprocessing methods—image resizing, sharpening, blurring, and foreground-background separation via K-means clustering—are systematically investigated and combined into a unified preprocessing pipeline to improve the performance of the open-source Tesseract OCR engine (v3.04) without relying on computationally expensive deep neural networks. Experimental evaluations conducted on five diverse categories of synthetic and real-world low-quality images (including digital television Electronic Program Guide screens) demonstrate that the proposed preprocessing pipeline elevates character recognition accuracy from a baseline of 70.2% to 92.9%.

Keywords—Optical Character Recognition; Image Preprocessing; Tesseract OCR; K-Means Clustering; Image Resizing; Sharpening; Binarization; Low Quality Images.

I. INTRODUCTION

Optical Character Recognition (OCR) enables automatic extraction of text from digitized documents, camera captures, and video frames [1]. Traditional OCR engines were originally architected for flatbed scanner inputs where text is printed in dark ink against a clean white page at spatial resolutions of 300 dots per inch (DPI) or higher [2]. However, emerging engineering applications—such as automated black-box testing of digital television sets, reading text from Electronic Program Guides (EPG), and extracting scene text from low-cost cameras—require recognizing text embedded in low-resolution images with arbitrary color combinations and compression noise [3].

When low-quality images are fed directly into an OCR engine such as Tesseract [1], internal global or local thresholding often merges adjacent characters, fractures thin character strokes, or misclassifies textured backgrounds as foreground glyphs [4]. Because deploying deep learning models is frequently impractical on resource-constrained embedded test benches or real-time inspection systems, classical digital image processing remains the primary mechanism for conditioning input images prior to recognition [5].

This paper proposes and evaluates a structured, four-stage image preprocessing pipeline tailored for improving OCR performance on degraded and low-resolution images. Specifically, we analyze the individual and combined effects of spatial scaling (bicubic resizing), edge enhancement (sharpening), noise suppression (Gaussian blurring), and color-space foreground-background separation using K-means clustering.

II. OVERVIEW OF THE TESSERACT OCR ARCHITECTURE

To design effective preprocessing filters, it is essential to understand how the underlying OCR engine processes pixel data. We utilize Tesseract OCR version 3.04, which operates purely on classical computer vision and pattern recognition algorithms [1]. The internal recognition architecture proceeds through six sequential stages:

1. **Adaptive Thresholding:** The input grayscale or color image is converted into a binary representation using Otsu's global thresholding or local adaptive thresholding [6]. If the background contains color gradients or compression artifacts, this initial step produces severe segmentation errors.
2. **Connected Component Analysis (CCA):** Outlines of foreground binary regions are extracted and grouped into blobs by analyzing topological nesting of outer contours and inner holes (e.g., inside characters such as 'O', 'B', or '8').
3. **Line and Word Finding:** Blobs are filtered by percentile height and fitted with quadratic spline baselines. Tesseract then segments text lines into discrete words by analyzing character pitch (fixed pitch versus proportional spacing) and inter-character gap histograms.
4. **Word Chopping and Association:** Joined or touching characters are split at concave vertices along polygonal outline approximations, while broken character fragments are re-associated using dynamic programming.
5. **Static and Adaptive Classification:** Polygonal features (short segments characterized by spatial position, direction, and length) are matched against a static font-independent prototype classifier. Words recognized with high confidence in the first pass train a font-sensitive adaptive classifier used in a second pass over the page.
6. **Linguistic Post-Processing:** Candidate character strings are validated against lexical dictionaries and character n-gram language models to resolve ambiguous glyphs (such as '1', 'l', and 'I').

III. PROPOSED IMAGE PREPROCESSING PIPELINE

Based on empirical failure analysis of the Tesseract pipeline on low-quality inputs, four primary image degradation factors were identified: insufficient character height in pixels, blurred character edges, high-frequency background noise, and low luminance contrast between colored text and colored backgrounds. We address these factors through a four-stage preprocessing pipeline.

A. Stage 1: Image Resizing (Spatial Interpolation)

Connected component analysis and polygonal feature extraction in Tesseract degrade sharply when the capital letter height falls below 20 pixels [1]. At small scales, quantization error along diagonal and curved strokes causes topological distortion. To restore sufficient spatial sampling density, the input image $I(x, y)$ of dimensions $M \times N$ is upsampled by a scaling factor s (where $s \in \{2, 3, 4\}$) to dimensions $(sM) \times (sN)$ using bicubic interpolation:

$$I_{resized}(x', y') = \sum_{i=-1}^{2} \sum_{j=-1}^{2} I(x_i, y_j) \cdot W(x' - x_i) \cdot W(y' - y_j) \quad (1)$$

where $W(u)$ denotes the bicubic convolution kernel. Unlike nearest-neighbor interpolation (which introduces staircase jaggedness along character contours) or bilinear interpolation (which excessively attenuates edge gradients), bicubic interpolation preserves smooth contour geometry for subsequent outline extraction.

B. Stage 2: Image Sharpening (Edge Enhancement)

Upsampling and optical defocus blur the transition zone between character strokes and the background. To steepen intensity gradients across character boundaries, an unsharp masking and Laplacian sharpening filter is applied to the resized image [7]. The sharpened image $I_{sharp}(x, y)$ is computed by subtracting a weighted second-order spatial derivative (Laplacian operator ∇^2) from the input intensity:

$$I_{sharp}(x, y) = I_{resized}(x, y) - \lambda \cdot [I_{resized}(x + 1, y) + I_{resized}(x - 1, y) + I_{resized}(x, y + 1) + I_{resized}(x, y - 1) - 4I_{resized}(x, y)] \quad (2)$$

where $\lambda > 0$ controls the sharpening gain. Enhancing high-frequency edge transitions prevents closely spaced characters from merging during binarization.

C. Stage 3: Image Blurring (Noise Suppression)

While sharpening accentuates character boundaries, it simultaneously amplifies high-frequency JPEG/MPEG block artifacts and impulse noise present in compressed video frames. To suppress isolated noise spikes without destroying stroke connectivity, a 2D Gaussian smoothing kernel $G_\sigma(u, v)$ with standard deviation σ (typically 3×3 window with $\sigma = 0.8$) is convolved with the image:

$$G_\sigma(u, v) = (1 / (2\pi\sigma^2)) \cdot \exp(-(u^2 + v^2) / (2\sigma^2)), \quad I_{smooth} = I_{sharp} * G_\sigma \quad (3)$$

Combining controlled sharpening followed by slight Gaussian smoothing regularizes stroke contours and eliminates micro-cavities along character edges.

D. Stage 4: Foreground-Background Separation via K-Means Clustering

Standard grayscale conversion followed by Otsu thresholding fails when foreground text and background regions have distinct chromaticity (hue/saturation) but similar luminance (e.g., red text on a green or blue background), or when the image contains inverted contrast (bright text on a dark background) [3], [8]. Furthermore, Tesseract performs optimally when text is rendered as black pixels (intensity 0) on a white background (intensity 255).

To achieve robust binarization independent of polarity and color combination, we formulate foreground-background separation as an unsupervised clustering problem in the 3D color space. Given a cropped text region containing P pixels with color vectors $\mathbf{c}_p = [R_p, G_p, B_p]^T$, K-means clustering partitions the pixels into $K = 2$ clusters (C_1 and C_2) with centroids $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$ by minimizing the within-cluster sum of squared Euclidean distances:

$$J = \sum_{k=1}^{2} \sum_{\mathbf{c}_p \in C_k} \|\mathbf{c}_p - \boldsymbol{\mu}_k\|^2 \quad (4)$$

Once convergence is reached, the algorithm must automatically determine which cluster corresponds to the background and which corresponds to the text foreground. In text bounding boxes, background pixels outnumber character stroke pixels and occupy the outer perimeter of the image patch. Therefore, the cluster containing the majority of pixels (and border pixels) is assigned to white (255), while the minority cluster is assigned to black (0). This guarantees a clean, polarity-normalized binary image prior to invoking Tesseract OCR.

IV. EXPERIMENTAL SETUP AND RESULTS

To evaluate the proposed preprocessing pipeline rigorously, a benchmark dataset of 250 test images was constructed and divided into five representative categories (50 images per category):

- **Set 1 (Font Types):** Text rendered in diverse serif, sans-serif, and decorative fonts at moderate resolution.
- **Set 2 (Small Font Sizes):** Low-resolution text where character heights range from 8 px to 16 px.
- **Set 3 (Colored Backgrounds):** Low luminance contrast and isoluminant colored text/background pairs, including bright text on dark backgrounds.
- **Set 4 (Noisy and Blurred Images):** Text degraded by additive Gaussian noise, salt-and-pepper noise, and JPEG compression artifacts.
- **Set 5 (Real Digital TV EPG Captures):** Real-world screen captures from digital television receivers containing multi-color GUI overlays, anti-aliased fonts, and video scaling distortion.

Recognition performance is quantified using Character Recognition Accuracy (CRA), defined through the Levenshtein edit distance between the OCR output string and the ground-truth text:

$$\text{CRA}\ (\%) = \max(0, [(N_{total} - (S + D + I)) / N_{total}]) \times 100\% \quad (5)$$

where N_{total} is the total number of ground-truth characters, and S, D, and I denote the number of character substitutions, deletions, and insertions, respectively.

Table I. Character Recognition Accuracy (%) Across Individual Preprocessing Methods and Combined Pipeline

Preprocessing Method	Set 1: Fonts	Set 2: Small Size	Set 3: Colored BG	Set 4: Noisy/Blur	Set 5: Real TV EPG	Overall Average
Baseline Tesseract OCR (No Preprocessing)	88.4%	54.6%	63.8%	68.2%	76.0%	**70.2%**
1. Image Resizing Only (Bicubic 3×)	91.2%	86.5%	69.1%	74.8%	83.4%	**81.0%**
2. Image Sharpening Only	89.6%	61.2%	66.4%	75.5%	79.8%	**74.5%**
3. Image Blurring Only (Gaussian 3×3)	87.9%	52.0%	65.0%	79.4%	77.2%	**72.3%**
4. K-Means Foreground-Background Separation Only	90.5%	60.8%	91.4%	76.1%	85.2%	**80.8%**
Resizing + Sharpening + Blurring	93.1%	89.2%	74.5%	86.8%	87.9%	**86.3%**
Proposed Full Pipeline (Resize + Sharpen + Blur + K-Means)	**95.8%**	**92.4%**	**94.6%**	**89.5%**	**92.2%**	**92.9%**

As shown in Table I, applying Tesseract OCR directly without preprocessing yields an average recognition accuracy of only 70.2%, with particularly poor performance on small font sizes (54.6%) and colored backgrounds (63.8%). Applying bicubic resizing (3×) alone dramatically boosts Set 2 accuracy from 54.6% to 86.5%, confirming that Tesseract's outline extractor requires sufficient pixel density per character. Conversely, K-means foreground-background separation resolves the failure modes in Set 3 (colored and inverted-polarity backgrounds), lifting accuracy from 63.8% to 91.4%.

Table II. Impact of Scaling Factor and Execution Time per Image Patch (Average Resolution 320 × 80 px)

Pipeline Configuration	Scaling Factor (s)	Preprocessing Time (ms)	Tesseract OCR Time (ms)	Total Execution Time (ms)	Overall Accuracy (%)
Baseline (No Preprocessing)	1.0×	0.0	42.5	42.5	70.2%
Proposed Pipeline (2× Upsampling)	2.0×	18.4	56.1	74.5	89.6%

Pipeline Configuration	Scaling Factor (s)	Preprocessing Time (ms)	Tesseract OCR Time (ms)	Total Execution Time (ms)	Overall Accuracy (%)
Proposed Pipeline (3× Upsampling - Optimal)	**3.0×**	**31.2**	**68.4**	**99.6**	**92.9%**
Proposed Pipeline (4× Upsampling)	4.0×	54.8	91.0	145.8	93.1%

Table II illustrates the trade-off between spatial scaling factor s, execution latency, and recognition rate. Increasing the scaling factor from 1× to 3× improves overall accuracy by +22.7 percentage points while maintaining total execution time under 100 ms per text region on a standard CPU, without requiring GPU acceleration. Scaling beyond 3× yields negligible accuracy gains (+0.2%) at the cost of a 46% increase in processing latency.

V. CONCLUSION

This paper presented a classical, non-deep-learning image preprocessing pipeline designed to enhance Optical Character Recognition on low-quality images. By sequentially combining bicubic image resizing, Laplacian sharpening, Gaussian noise smoothing, and K-means color clustering for foreground-background separation, the proposed framework overcomes the core limitations of internal OCR binarization and feature extraction. Experimental results across 250 synthetic and real-world digital TV images demonstrate an overall accuracy improvement from 70.2% to 92.9%, making the pipeline highly effective for real-time industrial and embedded text recognition applications.

REFERENCES

1. R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, vol. 2, Sep. 2007, pp. 629–633.
2. S. Mori, C. Y. Suen, and K. Yamamoto, "Historical review of OCR research and development," *Proceedings of the IEEE*, vol. 80, no. 7, pp. 1029–1058, Jul. 1992.
3. W. Bieniecki, S. Grabowski, and W. Rozenberg, "Image preprocessing for improving OCR accuracy," in *Proc. Int. Conf. Perspective Technologies and Methods in MEMS Design (MEMSTECH)*, Lviv-Polyana, Ukraine, May 2007, pp. 75–80.
4. O. D. Trier and A. K. Jain, "Goal-directed evaluation of binarization methods," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 17, no. 12, pp. 1191–1201, Dec. 1995.
5. J. Sauvola and M. Pietikäinen, "Adaptive document image binarization," *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, Feb. 2000.
6. N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, Jan. 1979.
7. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
8. J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.